Imagina

# Clean Architecture para Frontend con React y Node

Joan Cifo

Sesión 3 · Jueves 12 de noviembre de 2026 · 17:00 – 20:00

# Índice de la sesión

5 MIN

De reglas que protegen las capas a reglas de negocio que viven en un solo sitio

| Hora | Bloque | Contenido |
|---|---|---|
| 17:00 | **Apertura** | Repaso de la Sesión 2, ranking y repositorio imagina-clean-react-s3 |
| 17:15 | **Tema 5** | Arquitectura por features y Vertical Slice Architecture |
| 17:45 | **Tema 6** | Modelado de dominio, entidades y patrones de dominio |
| 18:30 | **Pausa** | 15 minutos |
| 18:45 | **Ejercicio** | Una sola regla de cancelación: value object, entidad con evento y API pública (45 min) |
| 19:30 | **Corrección** | Solución comentada, errores frecuentes y verificación antes/después |
| 19:45 | **Cierre** | Proyecto final · Test de conceptos · Feedback individual |

NOVEDAD Ejercicio más largo y exigente: 45 minutos en tres partes encadenadas (A → B → C). Cada parte usa lo que construye la anterior.

# Repaso relámpago y ranking

5 MIN

Cinco preguntas sobre la Sesión 2 · Responde en el chat todo a la vez: 1B, 2A…

DINÁMICA · 1 PUNTO POR ACIERTO

**1. En eslint.architecture.js, ¿qué capa puede importar composition?**
A) aplicación  B) presentación  C) dominio

**2. listUrgentOrders se probó sin red porque depende de…**
A) el puerto OrderReader  B) fetch  C) axios

**3. IFoo + FooImpl, una sola implementación y ningún límite que cruzar, es…**
A) Dependency Inversion  B) Liskov  C) ceremonia

**4. En la configuración plana de ESLint, si dos objetos definen la misma regla para un archivo…**
A) se combinan  B) gana el último

**5. El mapper que traduce ord_st a status vive en…**
A) infraestructura  B) dominio  C) presentación

### Ranking

Acumulado de dinámicas y tests. Se anuncia al inicio de cada sesión.

### Puntuable hoy

- Repaso (5)
- ¿Capas, feature o slice? (4)
- ¿Negocio, permiso o presentación? (3)
- ¿Entidad, VO, DTO o view model? (8)
- Test de conceptos (5)

CLAVE

Respuestas: 1B · 2A · 3C · 4B · 5A. Si fallaste la 4, revisa docs/solucion-s2.md, «Errores frecuentes».

# Repositorio de la Sesión 3

5 MIN

OrderDesk con la solución de ayer y el material de hoy

**PUESTA EN MARCHA**

```
# Terminal de Visual Studio Code (Ctrl+ñ)
$ git clone \
    https://github.com/joancifo/imagina-clean-react-s3.git
$ cd imagina-clean-react-s3
$ code .
$ npm install
$ npm test
  Test Files  22 passed | 4 skipped (26)
       Tests  72 passed | 27 skipped (99)

# Ejemplos y análisis de hoy
$ npm run demo:tema5    # 6 tests
$ npm run demo:tema6    # 20 tests
$ npm run boundaries
$ npm run rules
```

| Novedad | Para qué |
|---|---|
| `examples/tema5/approve-order/` | Un vertical slice completo |
| `examples/tema6/` | Value object, entidad, domain service y specification |
| `src/orders/domain/result.ts` | Errores previsibles como valores |
| `src/orders/index.ts` | API pública de la feature Pedidos |
| `docs/adr/001-estructura-pedidos.md` | Decisión de estructura (hito del proyecto) |

**PARTIDA** Incluye la solución de la Sesión 2: listUrgentOrders y las reglas de las cuatro capas ya están hechas. Los 27 tests omitidos son los del ejercicio.

# 05

# Arquitectura por features y Vertical Slice Architecture

Tema 5 · 17:15 – 17:45

Por tipo o por capacidad · Slices de extremo a extremo · Boundaries y API pública



Imagina

# Por tipo: un cambio, cinco carpetas

5 MIN

¿Dónde vive hoy la capacidad «Cancelar pedido»?

**EXPERIMENTO · GREP -RN CANCEL SRC**

```
# Organización actual: carpetas por TIPO de archivo
src/
├── components/OrderRow.tsx      botón + prop canCancel
├── pages/OrdersPage.tsx         regla 1 · diálogo · handler
├── pages/OrderDetailPage.tsx    regla 2 · botón
├── pages/DashboardPage.tsx      regla 3 · contador
├── services/api.ts              doAction(id, "cancel")
├── shared/utils.ts              trackEvent(…)
└── store/appStore.tsx           toast «Pedido cancelado»

# 7 archivos · 5 carpetas · 3 reglas distintas

$ npm run boundaries
  Pares de carpetas acopladas: 14
```

**1 Por tipo agrupa lo que se parece**

Todos los componentes juntos, todos los hooks juntos. Fácil de empezar.

**2 Pero los cambios llegan por capacidad**

Producto pide «cambiar la cancelación», no «cambiar los hooks».

**Medirlo**

npm run boundaries: tabla carpeta × carpeta. Cada celda con número es un acoplamiento.

**REFLEXIÓN** Por tipo agrupa lo que se parece; por feature agrupa lo que cambia junto. Es el Common Closure Principle de ayer aplicado a carpetas.

# Capas, feature modular o Vertical Slice

5 MIN

Tres formas de agrupar el mismo código · la regla de dependencia se mantiene en las tres

### CAPAS GLOBALES

```
src/
├── domain/
├── application/
├── infrastructure/
└── ui/
```

### FEATURE MODULAR

```
src/orders/
├── domain/
├── application/
├── infrastructure/
├── ui/
└── index.ts
```

### VERTICAL SLICE

```
src/orders/
├── approve-order/
│   ├── approve-order.ts
│   ├── ApproveButton.tsx
│   └── *.test.ts
└── cancel-order/
```

#### Qué es

Una carpeta por anillo para toda la aplicación.

#### Riesgo

Con varias features, cada capa las mezcla todas.

#### Qué es

Una carpeta por feature con capas dentro.

#### Riesgo

Más carpetas que en una app pequeña.

#### Qué es

Una carpeta por capacidad, de la UI a los datos.

#### Riesgo

Riesgo de duplicar reglas entre slices.

**RELACIÓN** Hexagonal (Cockburn, 2005), Onion (Palermo, 2008) y Clean (Martin, 2012) fijan la DIRECCIÓN de las dependencias; Vertical Slice (Bogard, 2018) fija cómo se AGRUPA el código.

# Anatomía de un vertical slice

5 MIN

examples/tema5/approve-order: «Aprobar pedido» de la interfaz a la red, en una carpeta

**ESTRUCTURA**

```
examples/tema5/approve-order/
├── index.ts                  API pública del slice
├── approve-order.ts          regla + comando + puerto
├── http-approval-gateway.ts  adapter: único que ve HTTP
├── ApproveOrderButton.tsx    interfaz
└── *.test.ts(x)              6 tests sin servidor
```

**EL COMANDO DEL SLICE**

```
export async function approveOrder(
  gateway: ApprovalGateway, order: Order, approver: Approver,
) {
  const decision = decideApproval(order, approver);
  if (decision.ok) await gateway.approve(order.id, approver.id);
  return decision;
}
$ npm run demo:tema5    ✓ 2 files · 6 tests
```

### 1 Comparte solo el dominio

Importa Order y calculateTotal de src/orders/domain. No importa otros slices.

### 2 La UI recibe el gateway por props

El test pasa un fake: { approve: async () => {} }. Sin mocks de módulos.

### 3 Con Node en el producto

El slice llega hasta POST /api/orders/:id/approve en server/index.mjs: entrada HTTP → handler → datos.

**IDEA** Prueba del slice: si se puede borrar la capacidad borrando una carpeta (y su línea en la composición), es un slice. Si no, es una feature repartida.

# Boundaries y API pública de una feature

5 MIN

Qué se comparte, qué no, y cómo se hace cumplir con ESLint

| Se puede compartir | No se comparte |
|---|---|
| Componentes base sin negocio (botón, tabla) | Reglas de una feature (canCancel) |
| Utilidades puras (formatCurrency) | Servicios genéricos (doRequest para todo) |
| Tipos comunes y contratos (Order, Result) | Tipos «de toda la app» (shared/types.ts) |
| Puertos transversales (logging, analítica) | Stores globales con datos de una pantalla |

**API PÚBLICA · SRC/ORDERS/INDEX.TS**

```
// src/orders/index.ts
export { canCancel } from './domain/cancellation';
export { toOrder } from './infrastructure/order-mapper';

// src/pages/OrdersPage.tsx
import { canCancel } from '../orders';            ✓
import { canCancel } from '../orders/domain/…';   ✗

$ npm run lint:boundaries
  '../orders/domain/cancellation' import is
  restricted from being used by a pattern.
  Usa Pedidos a través de su API pública:
  import { … } from "../orders"
```

**SHARED** Una carpeta shared sin dueño acaba en vertedero. Test rápido: cada archivo de shared debería poder nombrar a su propietario y a quién lo usa.

# ¿Capas, feature o slice?

5 MIN

Cuatro escenarios · Responde en el chat: 1A, 2B… · 1 punto por acierto

**DINÁMICA · A CAPAS · B FEATURE · C SLICE**

### 1 Escenario 1

Panel interno de 3 pantallas, una sola entidad, un desarrollador y cambios esporádicos.

### 2 Escenario 2

Backoffice con pedidos, clientes y facturación; cada área tiene reglas que comparten sus pantallas.

### 3 Escenario 3

Producto grande con 6 equipos; checkout, devoluciones y cupones tienen su UI, su API y su ritmo de despliegue.

### 4 Escenario 4

OrderDesk hoy: una feature principal, 3 pantallas y reglas compartidas (total, urgencia, cancelación).

### Criterios

- Número de features y de equipos
- ¿Las reglas se comparten entre pantallas o son de una capacidad?
- ¿Cada capacidad cambia y se despliega por su cuenta?
- CQRS (comandos y consultas separados) solo cuando lecturas y escrituras tienen modelos o ritmos distintos. No por defecto.

**CLAVE** Respuestas: 1A · 2B · 3C · 4B. Se valora el criterio: lo que decide es cómo llegan los cambios, no el gusto por una estructura.

# Migrar de tipos a features sin reescribir

5 MIN

Strangler Fig (M. Fowler, 2004): lo nuevo crece alrededor de lo viejo hasta sustituirlo

1. **Crear la feature junto al heredado** — src/orders · Sesión 2 ✓
2. **Publicar una API pública** — src/orders/index.ts · hoy
3. **Puente temporal para el heredado** — toOrder y toActor · hoy
4. **Mover pantallas a src/orders/ui** — Sesión 5
5. **Borrar el heredado y el puente** — Al final del proyecto

**PROMPT P1 · PROMPTS/SESION-3.MD**

```
#codebase Para la capacidad "Cancelar pedido",
lista TODOS los archivos que participan
(regla, botón, diálogo, llamada HTTP,
analítica, aviso) con ruta:línea y su
carpeta. Después compara cuántos archivos
y carpetas habría que tocar en: capas
globales, feature modular y Vertical Slice.
No propongas cambios todavía.

# Contrasta con npm run boundaries
# y con grep -rn cancel src
```

**DECISIÓN** OrderDesk: feature modular con capas internas; slices dentro de src/orders solo cuando una capacidad tenga UI, caso de uso y adapter propios. Se registra en el ADR-001.

# 06

# Modelado de dominio, entidades y patrones de dominio

Tema 6 · 17:45 – 18:30

Reglas escondidas · Value objects · Entidades e invariantes · Patrones opcionales



Imagina

# Caza de reglas escondidas

5 MIN

Reglas de negocio dentro de JSX, handlers, efectos y stores

**$ NPM RUN RULES · EXTRACTO**

```
# ubicación                  tipo         regla
OrdersPage.tsx:65            rol+estado   admin: pending|approved
OrdersPage.tsx:66            rol+umbral   manager: total < 5000
OrderDetailPage.tsx:31       rol+estado   admin: siempre
DashboardPage.tsx:26         rol+umbral   manager: total <= 6000
OrdersPage.tsx:92            rol+umbral   > 10000 → solo admin
OrdersPage.tsx:104           umbral       > 5000 → ir al detalle
OrdersPage.tsx:115           longitud     motivo ≥ 10 caracteres
OrdersPage.tsx:158           rol+umbral   > 20000 → riesgo crédito

# 29 candidatos · 19 en OrdersPage.tsx
# El script da CANDIDATOS: la clasificación es humana
```

**DINÁMICA · 1 PUNTO POR ACIERTO**

### N negocio · P permiso · V vista

1. Línea :104 · si el pedido supera 5000 €, navegar al detalle tras aprobar
2. Línea :158 · «Revisar riesgo de crédito» si lo pendiente supera 20000 €, solo para admin
3. Línea :115 · el motivo debe tener al menos 10 caracteres

**CLAVE** Responde 1V, 2NP… · Clave: 1V (decisión de UX) · 2NP (cuándo hay riesgo es negocio; quién lo ve, permiso) · 3N (motivo válido).

# Cinco modelos que no son lo mismo

5 MIN

Mismo pedido, cinco formas según quién lo usa y para qué

| Concepto | Qué lo define | En OrderDesk | Dónde vive |
|---|---|---|---|
| **Entidad** | Identidad y ciclo de vida | Order: P-1001 pasa de pending a approved y sigue siendo P-1001 | domain |
| **Value object** | Valor sin identidad, inmutable, igualdad por valor | CancellationReason, Money (examples/tema6) | domain |
| **DTO** | Forma de los datos en una frontera | OrderDto: { ord_id, cust_nm, ord_st… } | infrastructure |
| **Modelo de persistencia** | Forma en la que se almacena | Array orders de server/index.mjs · filtros en localStorage | infraestructura / servidor |
| **View model** | Datos preparados para una pantalla | UrgentOrder { id, customerName, total } · fila con rowColor | ui |

SMELL src/shared/types.ts mezcla tres en un solo tipo: Order lleva ord_st (DTO), total calculado (dominio) y rowColor y selected (view model).

# Value object: Money

5 MIN

examples/tema6/money.ts · cuando un number ya no expresa la intención

**MONEY.TS · EXTRACTO**

```ts
export type Money = Readonly<{ cents: number; currency: 'EUR' }>;

// Factory: única puerta de entrada
export function euros(amount: number): Money {
  if (!Number.isFinite(amount) || amount < 0)
    throw new RangeError(`Importe no válido: ${amount}`);
  return Object.freeze({ cents: Math.round(amount * 100),
                         currency: 'EUR' });
}
export const add = (a: Money, b: Money) =>
  fromCents(a.cents + b.cents);
export const equals = (a: Money, b: Money) =>
  a.cents === b.cents && a.currency === b.currency;
```

**EXPERIMENTO · CONSOLA DE NODE**

```
> 0.1 + 0.2 === 0.3
false
> equals(add(euros(0.1), euros(0.2)),
         euros(0.3))
true
$ npx vitest run examples/tema6/money
```

## Cuatro propiedades

- Sin identidad: dos euros(10) son iguales
- Inmutable: Object.freeze
- Igualdad por valor: equals, no ===
- Invariantes propias: no hay importes negativos

**MATIZ** Formatear «1.234,50 €» no es del value object: es presentación. Y un importe imposible es un error de programación (excepción); una entrada de usuario no válida es un Result.

# Tipos de TypeScript ≠ validación en ejecución

5 MIN

Los tipos se borran al compilar: la invariante la garantiza una factory

**BRANDED TYPE + FACTORY + RESULT**

```
// src/orders/domain/result.ts
type Result<T, E> = { ok: true; value: T }
                  | { ok: false; error: E };

// src/orders/domain/cancellation-reason.ts
type CancellationReason =
  string & { readonly __brand: 'CancellationReason' };

createCancellationReason(raw: string)
  : Result<CancellationReason, 'too-short' | 'too-long'>

cancelOrder(order, admin, 'sin validar', now);
✗ Argument of type 'string' is not assignable
  to parameter of type 'CancellationReason'
```

### 1 Datos de fuera: validar en la frontera

order-mapper.ts valida ord_st con oneOf (Sesión 2). Un «as Order» no comprueba nada en ejecución.

### 2 Valores del dominio: factory

Solo la factory puede hacer el cast al branded type. Si devuelve ok, la invariante se cumple.

### 3 Errores previsibles: Result

El error forma parte del tipo de retorno: el compilador obliga a tratarlo. Nada de alert ni throw.

**DOCS** TypeScript Handbook: las anotaciones de tipo desaparecen en el JavaScript emitido. Para validar datos externos a escala, schemas (Sesión 6).

# Entidad: identidad, ciclo de vida e invariantes

5 MIN

examples/tema6/order-entity.ts · el pedido es el mismo aunque cambien todos sus datos

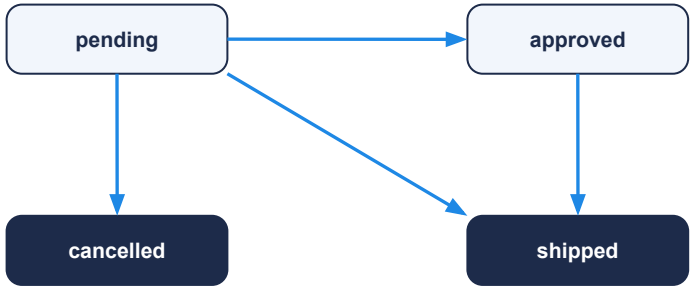
**ORDER-ENTITY.TS · EXTRACTO**

```ts
// Igualdad de ENTIDAD: por identidad
export const sameOrder = (a, b) => a.id === b.id;

// Factory: si devuelve ok, cumple las invariantes
createOrder(input): Result<Order, OrderError>
  // 'no-lines' · 'invalid-quantity' · 'duplicated-sku'

// Ciclo de vida como tabla
export const TRANSITIONS = {
  pending:   ['approved', 'cancelled'],
  approved:  ['shipped', 'cancelled'],
  shipped:   [],
  cancelled: [],
} satisfies Record<OrderStatus, OrderStatus[]>;
```

### Ciclo de vida del pedido




### Invariantes de un pedido

- Al menos una línea; cantidades enteras > 0; sin SKU repetidos
- Enviado y cancelado son estados finales
- Las transiciones devuelven un pedido nuevo

**IDEA** Una entidad no es «un objeto con id»: es algo cuyo ciclo de vida importa al negocio. Estados en oscuro: finales.

# Domain Service y Aggregate

5 MIN

Reglas que no son de una entidad · fronteras de consistencia

**DOMAIN SERVICE · CREDIT-RISK.TS**

```
// Depende de VARIOS pedidos, no de uno
export function needsCreditReview(
  orders: Order[],
): boolean {
  return pendingExposure(orders) > 20000;
}

// Quién VE el aviso (solo admin): permiso
// Cuándo HAY riesgo: negocio
```

**AGGREGATE ROOT · ORDER-ENTITY.TS**

```
// Las líneas solo cambian A TRAVÉS del pedido
export function changeQuantity(
  order: Order, sku: string, quantity: number,
): Result<Order, OrderError> {
  if (order.status !== 'pending')
    return err('not-editable');
  const lines = …;  // misma invariante que
  return checkLines(lines) …  // createOrder
}
```

### Cuándo un Domain Service

La regla necesita varias entidades o no encaja de forma natural en ninguna. Sin estado; función pura.

### Cuándo un Aggregate

Hay datos que deben ser consistentes A LA VEZ (pedido y sus líneas). Cliente y pedido no: aggregates separados.

**COSTE** Vernon (Implementing DDD, 2013): diseña aggregates pequeños. Uno grande bloquea cambios concurrentes y obliga a cargar datos que no necesitas.

# Specification y Domain Events: opcionales

5 MIN

Útiles en su contexto · ceremonia fuera de él

**SPECIFICATION · SPECIFICATION.TS**

```
const urgent = and(
  hasStatus('pending'),
  totalAbove(3000),
);
// Combinación nueva sin tocar las anteriores
const priorityForSales =
  and(urgent, isGoldCustomer);
```

**DOMAIN EVENT · CANCELLATION.TS**

```
type OrderCancelled = {
  type: 'OrderCancelled';  // pasado: ya ocurrió
  orderId: string;
  cancelledBy: string;
  reason: CancellationReason;
  occurredAt: string;      // ISO 8601
};
```

| Patrón | Aporta cuando… | Coste |
|---|---|---|
| **Specification** | Hay muchas combinaciones de criterios (filtros, segmentos, campañas) | Indirección: para una regla, isUrgent basta |
| **Domain Event** | Otras partes reaccionan (analítica, avisos, auditoría) sin acoplarse | Flujo menos visible; hay que decidir quién publica |

**NOMBRE** Un evento se nombra en pasado: OrderCancelled. CancelOrder sería un comando: una intención que aún puede fallar.

# Errores de dominio comprensibles

5 MIN

El dominio decide QUÉ falla; la interfaz decide CÓMO se cuenta

**DOMINIO · CÓDIGOS ESTABLES**

```
type CancellationDenied =
  | 'no-session'
  | 'already-closed'
  | 'forbidden'
  | 'over-manager-limit';

checkCancellation(order, actor)
  : Result<Actor, CancellationDenied>

// En logs: el código es estable y buscable
// log.warn('cancel.denied', { code, orderId })
```

**PRESENTACIÓN · UI/CANCELLATION-MESSAGES.TS**

```
export const CANCELLATION_MESSAGES = {
  'too-short': 'El motivo debe tener…',
  'already-closed': 'Este pedido ya está…',
  // …uno por código
} satisfies Record<
  ReasonError | CancellationDenied, string>;
```

| Estado de negocio (dominio) | Estado visual (ui) |
|---|---|
| status del pedido, motivo | saving, compact, toast |
| ¿Se puede cancelar? | ¿Está abierto el diálogo? |

**REGLA** alert() en un handler no es gestión de errores. Código estable en el dominio + mensaje en la UI + satisfies para no olvidar ninguno.

# ¿Entidad, value object, DTO o view model?

5 MIN

Ocho piezas reales de OrderDesk · 1 punto por acierto

DINÁMICA · E · V · D · M

1. **Pedido P-1001, que pasa de pending a approved**
2. **{ ord_id, cust_nm, ord_st } de GET /api/orders**
3. **Motivo de cancelación ya validado**
4. **{ id, customerName, total } para la lista de urgentes**
5. **Importe «1.234,50 €» como cantidad con moneda**
6. **Usuario u2 (Pau), con rol manager**
7. **Fila de la tabla con rowColor: 'grey'**
8. **{ usr_id, usr_nm, usr_role } de GET /api/me**

CLAVE

Respuestas: 1E · 2D · 3V · 4M · 5V · 6E · 7M · 8D. Prompt P2 (prompts/sesion-3.md): pide a Copilot la misma clasificación y compárala.

# 15

## Pausa ·15 minutos

Volvemos a las 18:45 con el ejercicio
Para pensar: ¿qué regla de negocio de tu proyecto real vive hoy dentro de un componente?

# Ejercicio: una sola regla de cancelación

5 MIN

docs/ejercicio-s3.md · 45 minutos en tres partes encadenadas · rama s3/<tu-nombre>

| Pantalla | Regla actual | Problema |
|---|---|---|
| OrdersPage | admin: pendiente o aprobado · manager: pendiente y < 5000 € | La única coherente |
| OrderDetailPage | admin: siempre · manager: si no está enviado | Admin cancela enviados y cancelados |
| DashboardPage | admin: si no está enviado · manager: ≤ 6000 € sin descuento | Otro límite y otro total |

**ANTES DE EMPEZAR**

```
$ git checkout -b s3/<tu-nombre>
$ npm test     # 72 passed | 27 skipped
# A 18:45-19:00 · B 19:00-19:15 · C 19:15-19:30
```

## Regla oficial (decisión de producto)

- Sin sesión, nadie cancela
- Enviado o cancelado = cerrado: no se cancela nunca
- Admin: pendientes y aprobados
- Manager: pendientes con total (descuento e IVA) menor de 5000 €
- Agente: no cancela
- Motivo obligatorio: entre 10 y 280 caracteres tras quitar espacios sobrantes

**OBJETIVO** Al terminar: 1 regla en el dominio, 3 pantallas que la usan, un value object, un evento y una frontera que ESLint hace cumplir.

# Parte A ·Value object CancellationReason

10 MIN

18:45 – 19:00 · src/orders/domain/cancellation-reason.ts

## Pasos

1. Quita .skip en cancellation-reason.test.ts → 5 tests en rojo
2. Implementa createCancellationReason: trim(), colapsa espacios (/\s+/g → " ") y valida longitud con REASON_MIN_LENGTH y REASON_MAX_LENGTH
3. Devuelve err('too-short'), err('too-long') u ok(texto as CancellationReason)
4. En src/orders/index.ts (bloque «Parte A») exporta createCancellationReason, sus tipos y CANCELLATION_MESSAGES
5. En OrdersPage.tsx (handleCancel) sustituye reason.trim().length < 10 por la factory; si falla: alert(CANCELLATION_MESSAGES[parsed.error]). Importa solo desde '../orders'
6. Quita .skip del bloque «Parte A» en src/pages/cancel-order.test.tsx

**PUNTO DE PARTIDA**

```
// Ya está hecho (no lo cambies):
type CancellationReason =
  string & { readonly __brand: 'CancellationReason' };
type ReasonError = 'too-short' | 'too-long';
REASON_MIN_LENGTH = 10  · REASON_MAX_LENGTH = 280

// Lo implementas tú:
createCancellationReason(raw: string)
  : Result<CancellationReason, ReasonError>

$ npm test
✓ CancellationReason (5)
✓ Parte A · el motivo se valida… (1)
```

**PISTA** El test de pantalla escribe 'a b': 12 caracteres con trim(), 3 después de normalizar. Por eso normalizar va antes de validar.

# Parte B ·Regla oficial y transición con evento

15 MIN

19:00 – 19:15 · src/orders/domain/cancellation.ts

## Pasos

1. Quita .skip en los dos describe de cancellation.test.ts
2. checkCancellation(order, actor) en este orden: no-session → already-closed → admin ok → manager (pendiente y calculateTotal(order) < MANAGER_CANCEL_LIMIT, si no over-manager-limit) → forbidden. Éxito: ok(actor)
3. canCancel(order, actor): booleano para la UI. Delega; no repitas la regla
4. cancelOrder(order, actor, reason, now): mismo error si falla; si no, { order, event } con un pedido NUEVO en cancelled y el evento OrderCancelled
5. npm test && npm run typecheck && npm run lint:arch

**CONTRATO (YA DEFINIDO)**

```
type Actor = { id: string; role: Role };
type CancellationDenied = 'no-session'
  | 'already-closed' | 'forbidden'
  | 'over-manager-limit';

checkCancellation(order, actor)
  : Result<Actor, CancellationDenied>
canCancel(order, actor): boolean
cancelOrder(order, actor, reason, now)
  : Result<{ order; event: OrderCancelled },
           CancellationDenied>

✓ regla oficial (10) · cancelOrder (4)
```

**TRAMPAS** El test congela el pedido (Object.freeze): mutarlo falla. «Menor de 5000 €» es <, no <=. El total es calculateTotal(order), con descuento e IVA.

# Parte C ·API pública y una sola regla

15 MIN

19:15 – 19:30 · src/orders/index.ts · eslint.boundaries.js · src/pages

## Pasos

1. index.ts (bloques «Parte C»): exporta canCancel, cancelOrder y sus tipos, y el puente temporal toOrder (infrastructure/order-mapper) y toActor (infrastructure/actor-mapper)
2. Sustituye las tres reglas por canCancel(toOrder(o), toActor(user)): canCancelOrder en OrdersPage; la constante canCancel en OrderDetailPage (renómbrala: cancellable); cancellable en DashboardPage (filtra orders, no rows)
3. eslint.boundaries.js: el heredado no puede importar **/orders/** (sí ../orders)
4. Quita .skip en boundaries.test.ts y en el bloque «Parte C» de cancel-order.test.tsx

**COMPROBACIÓN FINAL Y ENTREGA**

```
$ npm test
  Tests  99 passed (99)
$ npm run typecheck
$ npm run lint:arch
$ npm run lint:boundaries
$ npm run rules
  # antes: 29 candidatos · después: 23

$ git add -A
$ git commit -m "feat(orders): regla oficial
  de cancelación, value object y API pública"
$ git push -u origin s3/<tu-nombre>
```

**AMPLIACIÓN** Usa needsCreditReview en el aviso de riesgo de OrdersPage · Prompt P3 para extraer la regla de aprobación · Rellena docs/adr/001.

# Solución: value object y regla oficial

5 MIN

docs/solucion-s3.md en el repositorio de solución

**PARTE A · CANCELLATION-REASON.TS**

```
export function createCancellationReason(
  raw: string,
): Result<CancellationReason, ReasonError> {
  const text = raw.trim().replace(/\s+/g, ' ');
  if (text.length < REASON_MIN_LENGTH)
    return err('too-short');
  if (text.length > REASON_MAX_LENGTH)
    return err('too-long');
  return ok(text as CancellationReason);
}

// OrdersPage.handleCancel
const parsed = createCancellationReason(reason);
if (!parsed.ok) {
  alert(CANCELLATION_MESSAGES[parsed.error]); return;
}
```

**PARTE B · CANCELLATION.TS**

```
if (actor === null) return err('no-session');
if (order.status === 'shipped' ||
    order.status === 'cancelled')
  return err('already-closed');
if (actor.role === 'admin') return ok(actor);
if (actor.role === 'manager' && order.status === 'pending')
  return calculateTotal(order) < MANAGER_CANCEL_LIMIT
    ? ok(actor) : err('over-manager-limit');
return err('forbidden');

// cancelOrder
const check = checkCancellation(order, actor);
if (!check.ok) return check;
return ok({ order: { ...order, status: 'cancelled' },
  event: { type: 'OrderCancelled', orderId: order.id,
    cancelledBy: check.value.id, reason,
    occurredAt: now.toISOString() } });
```

**COMENTAR** Normalizar antes de validar · el cast solo en la factory · already-closed antes que el rol · ok(actor) evita actor! · now por parámetro: test determinista.

# Solución: API pública y pantallas

5 MIN

Errores frecuentes detectados en la corrección

**PARTE C**

```
// src/orders/index.ts
export { canCancel, cancelOrder, … } from './domain/cancellation';
export { toOrder } from './infrastructure/order-mapper';
export { toActor } from './infrastructure/actor-mapper';

// eslint.boundaries.js
{ files: LEGACY, rules: { 'no-restricted-imports': ['error',
  { patterns: [{ group: ['**/orders/**'], message: … }] }] } }

// OrdersPage · OrderDetailPage · DashboardPage
const actor = toActor(user);
const canCancelOrder = (o) => canCancel(toOrder(o), actor);
const cancellable = canCancel(toOrder(order), toActor(user));
orders.filter((o) => canCancel(toOrder(o), actor));
```

## Errores frecuentes

- const canCancel = canCancel(…) en el detalle: la constante oculta la función importada; TypeScript: «used before its declaration»
- Filtrar rows en el panel: sin cust_tier ni líneas, toOrder no puede construir el pedido
- Importar '../orders/domain/cancellation' «porque funciona»: lint:boundaries lo rechaza
- Patrón **/orders sin /**: bloquea también ../orders
- Actualizar el snapshot de OrdersPage sin preguntarse por qué cambió (no debe cambiar)

**PUENTE** toOrder y toActor en la API pública son TEMPORALES y están marcados como tales: desaparecen al migrar las pantallas a src/orders/ui (Sesión 5).

# Antes y después, con evidencia

5 MIN

Cada fila se comprueba con un comando o un test

| Métrica | Antes | Después | Evidencia |
|---|---|---|---|
| Reglas de cancelación | 3 | 1 | cancellation.ts |
| Candidatos a regla fuera del dominio | 29 | 23 | npm run rules |
| Admin cancela un pedido enviado (detalle) | Sí | No | cancel-order.test |
| Cancelables para admin en el panel | 5 | 4 | cancel-order.test |
| Imports al interior de src/orders | Libres | Prohibidos | lint:boundaries |
| Tests en verde | 72 | 99 | npm test |

## Review del Pull Request

- Bloque «Dominio» de la plantilla: cada casilla con un comando
- ¿Qué comportamiento ha cambiado a propósito?
- ¿Quién lo decidió? (producto, en el enunciado)
- ¿Qué test lo fija?

**PREGUNTA** El ejercicio ha cambiado el comportamiento de dos pantallas. ¿Cómo lo comunicarías en la descripción del Pull Request?

# Proyecto final: hito de la Sesión 3

5 MIN

Feature Pedidos de OrderDesk, transformada de forma incremental sesión a sesión

| S1 | S2 | S3 | S4 | S5 |
|---|---|---|---|---|
| Diagnóstico y matriz | Reglas, checklist y capas | Estructura y reglas puras | Casos de uso, puertos y Composition Root | Hooks, view models, estado y routing |

## Hecho hoy

- Regla oficial de cancelación en el dominio, con 10 casos probados
- Value object CancellationReason y evento OrderCancelled
- API pública de Pedidos protegida por lint:boundaries
- Tres pantallas con una sola regla

**COMPLETA EL HITO**

```
# Antes de la Sesión 4
$ code docs/adr/001-estructura-pedidos.md
# Decisión · Consecuencias ·
# Lo que decidimos NO hacer

# Evidencias del contexto:
$ npm run boundaries
$ npm run rules
```

**LUNES** Sesión 4: casos de uso, puertos, adapters, fakes y Composition Root. El evento OrderCancelled necesitará quien lo publique.

# Test de conceptos: 5 preguntas, 30 segundos

5 MIN

Individual · Responde en el chat al final: 1A, 2C… · 1 punto por acierto

**1. Vertical Slice Architecture agrupa el código por…**
A) capa técnica  B) capacidad de extremo a extremo  C) tipo de archivo

**2. Dos value objects son iguales cuando…**
A) tienen el mismo id  B) son la misma referencia  C) tienen el mismo valor

**3. Escribir 'texto' as CancellationReason fuera de la factory…**
A) se salta la invariante  B) valida en ejecución  C) es necesario para compilar

**4. OrderCancelled es…**
A) un comando  B) un evento de dominio  C) un DTO

**5. ¿Por qué cancelOrder recibe now: Date en lugar de llamar a new Date()?**
A) por rendimiento  B) determinismo: el reloj es un detalle externo  C) lo exige TypeScript

RANKING La clave se da al terminar la ronda. Los puntos se suman a las dinámicas de hoy; el ranking se anuncia al inicio de la Sesión 4.

# Feedback individual y próxima sesión

5 MIN

Lunes 16 de noviembre · 17:00 – 20:00 · Casos de uso, Ports & Adapters y Composition Root

FEEDBACK · MENSAJE PRIVADO EN EL CHAT

## Envíame en privado

- La idea que te llevas de hoy
- El concepto que quieres repasar el lunes
- Tu Pull Request s3 recibirá un comentario individual

## Tres ideas de hoy

- Agrupa el código por cómo llegan los cambios; protege la frontera con una API pública
- Value object + factory + Result: la invariante se cumple o no compila
- Una regla de negocio vive en un sitio; las pantallas la consultan

## Antes de la Sesión 4

- Completa docs/adr/001-estructura-pedidos.md
- Sube tu rama s3/<tu-nombre> y abre el Pull Request
- npm test, typecheck y los dos lint en verde

## Sesión 4 · 16 de noviembre

- Repaso
- Casos de uso y lógica de aplicación
- Ports & Adapters, inyección de dependencias y Composition Root

CONTINUIDAD El lunes, OrderCancelled necesitará quien lo publique y la fecha llegará de un puerto Clock: el dominio de hoy no cambiará.